

✓ Lab 3 : BP Data Cleaning using R

Dataset: Heart Disease dataset

```
heart <- read.csv("heart.csv")
str(heart)
```

```
'data.frame':  1025 obs. of  14 variables:
 $ age      : int  52 53 70 61 62 58 58 55 46 54 ...
 $ sex      : int  1 1 1 1 0 0 1 1 1 1 ...
 $ cp       : int  0 0 0 0 0 0 0 0 0 0 ...
 $ trestbps: int  125 140 145 148 138 100 114 160 120 122 ...
 $ chol     : int  212 203 174 203 294 248 318 289 249 286 ...
 $ fbs      : int  0 1 0 0 1 0 0 0 0 0 ...
 $ restecg  : int  1 0 1 1 1 0 2 0 0 0 ...
 $ thalach  : int  168 155 125 161 106 122 140 145 144 116 ...
 $ exang     : int  0 1 1 0 0 0 0 1 0 1 ...
 $ oldpeak  : num  1 3.1 2.6 0 1.9 1 4.4 0.8 0.8 3.2 ...
 $ slope    : int  2 0 0 2 1 1 0 1 2 1 ...
 $ ca       : int  2 0 0 1 3 0 3 1 0 2 ...
 $ thal     : int  3 3 3 3 2 2 1 3 3 2 ...
 $ target   : int  0 0 0 0 0 1 0 0 0 0 ...
```

```
set.seed(123)
n <- nrow(heart)

neg_index <- sample(1:n, 5)
na_index <- sample(setdiff(1:n, neg_index), 5)
high_index <- sample(setdiff(1:n, c(neg_index, na_index)), 5)

heart$trestbps[neg_index] <- -heart$trestbps[neg_index]
heart$trestbps[na_index] <- NA
heart$trestbps[high_index] <- sample(305:350, 5, replace = TRUE)

write.csv(heart, "heart_dirty_data.csv", row.names = FALSE)
```

```
clean_bp <- function(x){
  if(is.na(x)){
    return(NA)
  }
  if(x < 0){
    return(NA)
  }
  if(x > 250){
    return(250)
  }
  return(x)
}

heart$trestbps_clean <- sapply(heart$trestbps, clean_bp)
heart[c(neg_index, na_index, high_index), c("trestbps", "trestbps_clean")]
```

A data.frame: 15 × 2

	trestbps	trestbps_clean
	<int>	<dbl>
415	-170	NA
463	-118	NA
179	-110	NA
526	-130	NA
195	-160	NA
943	NA	NA
823	NA	NA
118	NA	NA
301	NA	NA
231	NA	NA
248	331	250
14	309	250
379	331	250
673	332	250
610	313	250

```
safe_mean <- function(x){
  result <- tryCatch({
    mean(x, na.rm = TRUE)
  }, error = function(e){
    print("error in mean calculation")
    NA
  })
  return(result)
}

safe_mean(heart$trestbps)
```

131.2

```
safe_ratio <- function(a, b){
  result <- tryCatch({
    if(is.na(a) | is.na(b)){
      stop("missing value")
    }
    if(b == 0){
      stop("cannot divide by zero")
    }
    a / b
  }, error = function(e){
    print(paste("error:", e$message))
    NA
  })
  return(result)
}

safe_ratio(200, 0)
safe_ratio(200, NA)
safe_ratio(NA, 100)
safe_ratio(200, 100)
```

```
[1] "error: cannot divide by zero"
<NA>
[1] "error: missing value"
<NA>
[1] "error: missing value"
<NA>
2
```

```
heart$ratio <- heart$chol / heart$trestbps_clean
head(heart$ratio, 10)
```

```
1.696 · 1.45 · 1.2 · 1.37162162162162 · 2.1304347826087 · 2.48 · 2.78947368421053 · 1.80625 · 2.075 · 2.34426229508197
```

```
bp <- heart$trestbps

loop_clean <- function(x){
  output <- c()
  for(i in 1:length(x)){
    val <- x[i]
    if(is.na(val)){
      output[i] <- NA
    } else if(val < 0){
      output[i] <- NA
    } else if(val > 250){
      output[i] <- 250
    } else {
      output[i] <- val
    }
  }
  return(output)
}

vector_clean <- function(x){
  x[!is.na(x) & x < 0] <- NA
  x[!is.na(x) & x > 250] <- 250
  return(x)
}

result1 <- loop_clean(bp)
result2 <- vector_clean(bp)

identical(result1, result2)
```

```
TRUE
```

```
t1 <- system.time(for(i in 1:500){ loop_clean(bp) })
t2 <- system.time(for(i in 1:500){ vector_clean(bp) })
```

```
t1
t2
```

```
user system elapsed
0.309  0.013  0.323
user system elapsed
0.02   0.00   0.02
```

```
heart$trestbps_clean <- result2
```

```
sum(is.na(heart$trestbps_clean))
min(heart$trestbps_clean, na.rm = TRUE)
max(heart$trestbps_clean, na.rm = TRUE)
mean(heart$trestbps_clean, na.rm = TRUE)
median(heart$trestbps_clean, na.rm = TRUE)

any(heart$trestbps_clean < 0, na.rm = TRUE)
any(heart$trestbps_clean > 250, na.rm = TRUE)
```

```
10
94
250
132.16354679803
130
FALSE
FALSE
```

```
heart$trestbps <- heart$trestbps_clean
heart$trestbps_clean <- NULL

write.csv(heart, "cleaned_heart_data.csv", row.names = FALSE)
```

Conclusion: The vectorized method ran faster than the for loop, since R does not have to repeat the same instruction one element at a time. After cleaning, trestbps has no negative values and nothing above 250, and the 10 missing readings are still marked as NA.